

Turbo Combat Racing

SPEED • COMBAT • HEAT

A browser-based arcade combat racing game built with HTML5 Canvas and plain JavaScript. This document covers the game concept, technical architecture, AI and heat logic, UI and deployment, and the expansion roadmap.

Section	Topic
1	Game concept and core mechanics
2	Technical architecture and rendering engine
3	AI and logic (heat and enemy)
4	UI/UX and platform deployment
5	Roadmap and future enhancements

1. Game Concept and Core Mechanics

Turbo Combat Racing is an arcade racer where three systems feed each other in a loop: **speed**, **combat** and **resource management** (health and heat). Boosting makes you faster but builds heat; overheating slows and damages you; combat forces you to choose between driving the racing line and lining up a shot.

Vehicles

Car	HP	Top speed	Armor	Heat rate	Character
F40	471	324 km/h	100	22	Balanced all-rounder
Aventador	700	350 km/h	90	30	Fastest, runs hottest, fragile
911 GT3	502	318 km/h	110	18	Best handling and cooling

HP and top speed are approximate real-world figures shown on the selection screen. In-game tuning (max speed, acceleration, handling) is in `js/cars.js`.

Race rules

- 3 laps against 3 AI rivals; first across the line wins, and your finishing place is shown on the result screen.
- Boost (Shift) raises top speed by 40% and acceleration by 80% but builds heat.
- Space fires a bullet (8 damage, 0.22 s cooldown, 30 starting ammo).
- Leaving the track cuts your top speed to 45%.
- Ramming costs health once per impact, scaled by the speed difference.
- Pickups respawn every 8 s: **ammo** (+15), **repair** (+25 health), **coolant** (-40 heat).
- You lose if your health reaches zero.

Controls

Key	Action
W / Up	Accelerate
S / Down	Brake and reverse
A, D / Left, Right	Steer
Shift	Boost
Space	Shoot

2. Technical Architecture and Rendering Engine

The game is a static site with no dependencies and no build step. It uses an HTML5 Canvas for the world and CSS3 for the HUD and menus.

Modules

File	Responsibility
index.html	Pages: home, car select, game, result, about, contact
css/style.css	Neon-glow theme, HUD bars, car cards
js/cars.js	Global config and car definitions
js/collision.js	Geometry helpers, track shape, circle and AABB collision
js/heat.js	Heat formula
js/ai.js	Rival driving and predictive auto-combat
js/game.js	Game loop, update(), render()
js/ui.js	Navigation, car selection, result screen

Game loop

A fixed time step of 1/60 s drives `update()`. Frames from `requestAnimationFrame` accumulate elapsed time and run as many fixed steps as needed, then `render()` draws once. This keeps physics identical on 60 Hz and 144 Hz screens, and it is the foundation for deterministic multiplayer later.

```
each frame: acc += dt
while acc >= 1/60: update(1/60)
render()
```

Collision detection

- **Circle to circle**: car versus car (push apart, share speed, damage on real impacts) and bullet versus car.
- **AABB**: car versus pickup, and a cheap broad-phase check before the circle test for bullets.
- Track bounds use two ellipse inequalities: inside the outer ellipse and outside the inner one.

Rendering and HUD

The canvas draws the track (even-odd filled ring with glowing edges), pickups, sparks, bullets and cars using `shadowBlur` for the neon look. The HUD above the canvas is plain HTML styled with CSS3 box and text shadows: lap, speed in km/h, health bar, heat bar and ammo.

3. AI and Logic (Heat and Enemy)

Heat management

Every frame, each car's heat is updated with:

$$Heat_{final} = Heat_{initial} + (Boost_{time} \times Rate) - (Cooling_{constant})$$

- **$Boost_{time}$** is the frame time while boost is held, otherwise 0.
- **Rate** is the car's heat per second of boosting (F40 22, Aventador 30, 911 GT3 18).
- **$Cooling_{constant}$** removes 12 heat per second (scaled by frame time).
- Heat is clamped between 0 and 100.
- At 100 the engine **overheats**: boost locks, top speed drops 40%, and the car loses 2 health per second. Normal operation resumes when heat falls to 40.

Because cooling is constant, a car can boost indefinitely only if its boost duty cycle stays below Cooling / Rate: about 55% for the F40, 40% for the Aventador and 67% for the 911 GT3.

Enemy driving

Each rival chases a point a short distance ahead on the racing line (a lead of 0.32 rad around the track ellipse, shifted into its own lane). Steering is proportional to the heading error, and throttle eases off in sharp turns. Rivals manage their own heat with hysteresis: they start boosting below 40 heat and stop above 75, and only boost when roughly pointed down the straight.

Predictive tracking and auto-combat

A bullet needs time to arrive, so a rival aims at where the player will be, not where the player is:

```
t = distance / bulletSpeed
predicted = playerPosition + playerVelocity x t
```

A rival fires automatically when the player is within 380 px and the predicted aim lies inside a 0.5 rad cone off the car's nose. Aim error grows as the rival's skill drops (0.92, 0.82 and 0.72 for the three rivals), and each rival's top speed is scaled by the same skill value.

4. UI/UX and Platform Deployment

Screens

Screen	Contents
Home	Cover art, tagline, **START RACE** button, top navigation (Home, About, Contact)
Car select	Three cards with HP, top speed and stat bars; GO button enables once a car is chosen
Game	Canvas with 3-2-1 countdown, HUD (lap, speed, health, heat, ammo), position and timer
Result	Victory, finishing place or wrecked, with a Race Again button
About / Contact	Project description and GitHub link

Deployment

The folder is already modular and uses only relative paths, so it deploys unchanged to any static host.

Platform	Steps
GitHub Pages	Push to a repository, then Settings > Pages > branch main, root folder.
Netlify	Drag the folder onto app.netlify.com/drop, or connect the repo with an empty build command and publish directory '.'.
itch.io	Zip the folder contents so index.html is at the zip's top level, upload as an HTML project and tick 'play in browser'.

See docs/deployment.md in the project for details.

5. Roadmap and Future Enhancements

Phase	Focus	Key work
1	Sound (Web Audio API)	Engine tone tied to speed, boost, shot, hit and explosion effects synthesised with oscillators and noise, overheat warning beep, volume and mute control.
2	Procedural tracks	Random closed splines, distance-to-centerline track test, AI that follows the generated racing line, shareable seed.
3	Multiplayer and learned AI	WebRTC peer-to-peer play with input exchange on the existing fixed-step simulation, TensorFlow.js driving and aiming model trained on recorded laps, leaderboards and ghost replays.

Verification note

The engine files were syntax-checked and run headless in Node: the simulation runs without errors and AI rivals complete three laps. Visual rendering and keyboard feel should be confirmed by playing in a browser.